Computer Architecture
The Processor

Tatpong Katanyukul

CoE, KKU

19 เมษายน พ.ศ. 2558

Introduction

- CPU performance factors
  - Instruction count

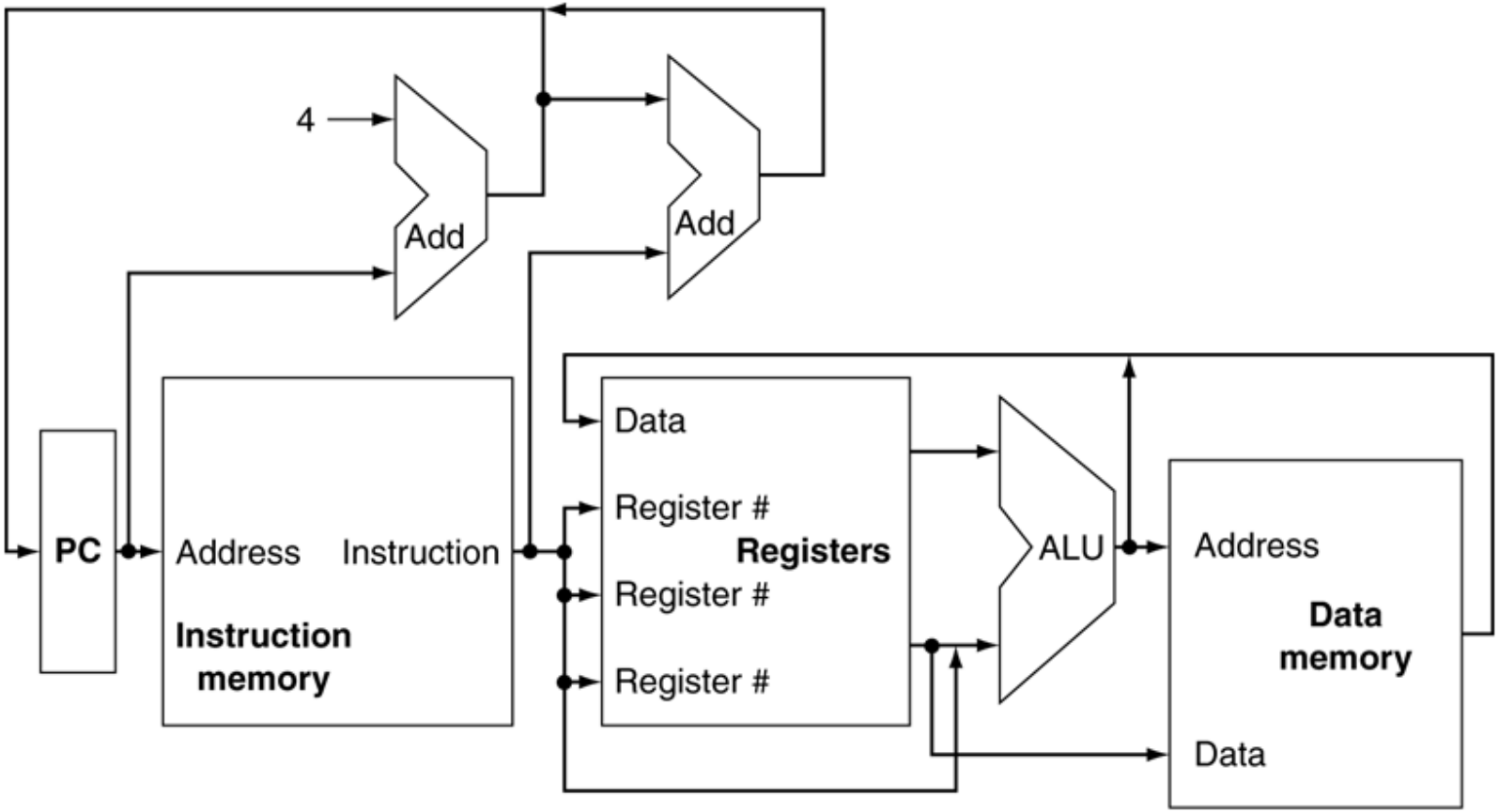
    Determined by ISA and compiler
  - CPI and Cycle time

    Determined by CPU hardware
- We will examine simplified MIPS implementation
- Simple subset shows most aspects
  - Memory reference: lw, sw
  - Arithmetic/logical: add, sub, and, or, slt
  - Control transfer: beq, j

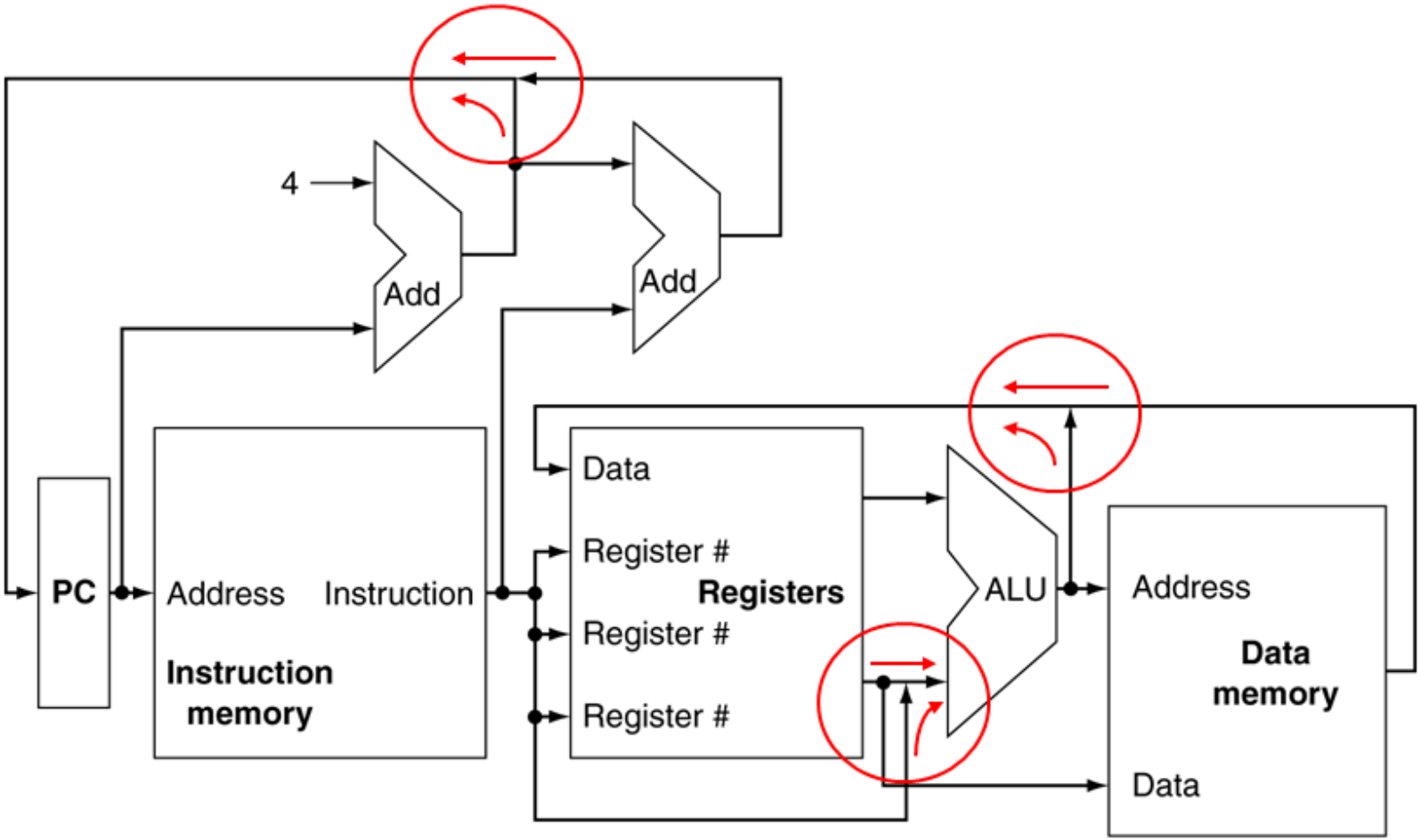
Instruction Execution

- PC $\rightarrow$ instruction memory, fetch instruction
- Register numbers $\rightarrow$ register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC $\leftarrow$ PC + 4 or target address

Simplified CPU Overview: First Glimpse

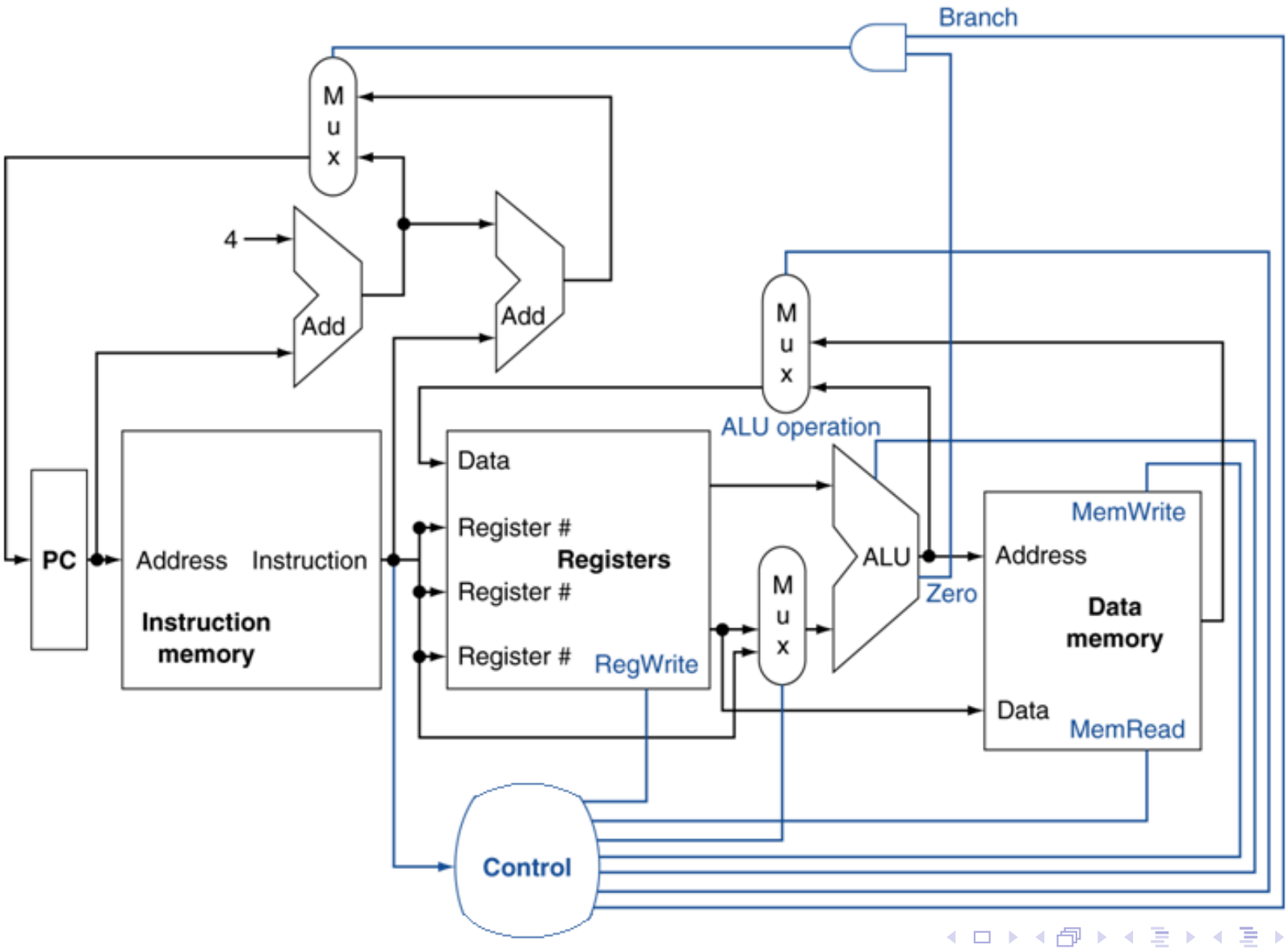


Simplified CPU Overview: Multiplexers As Key Controllers

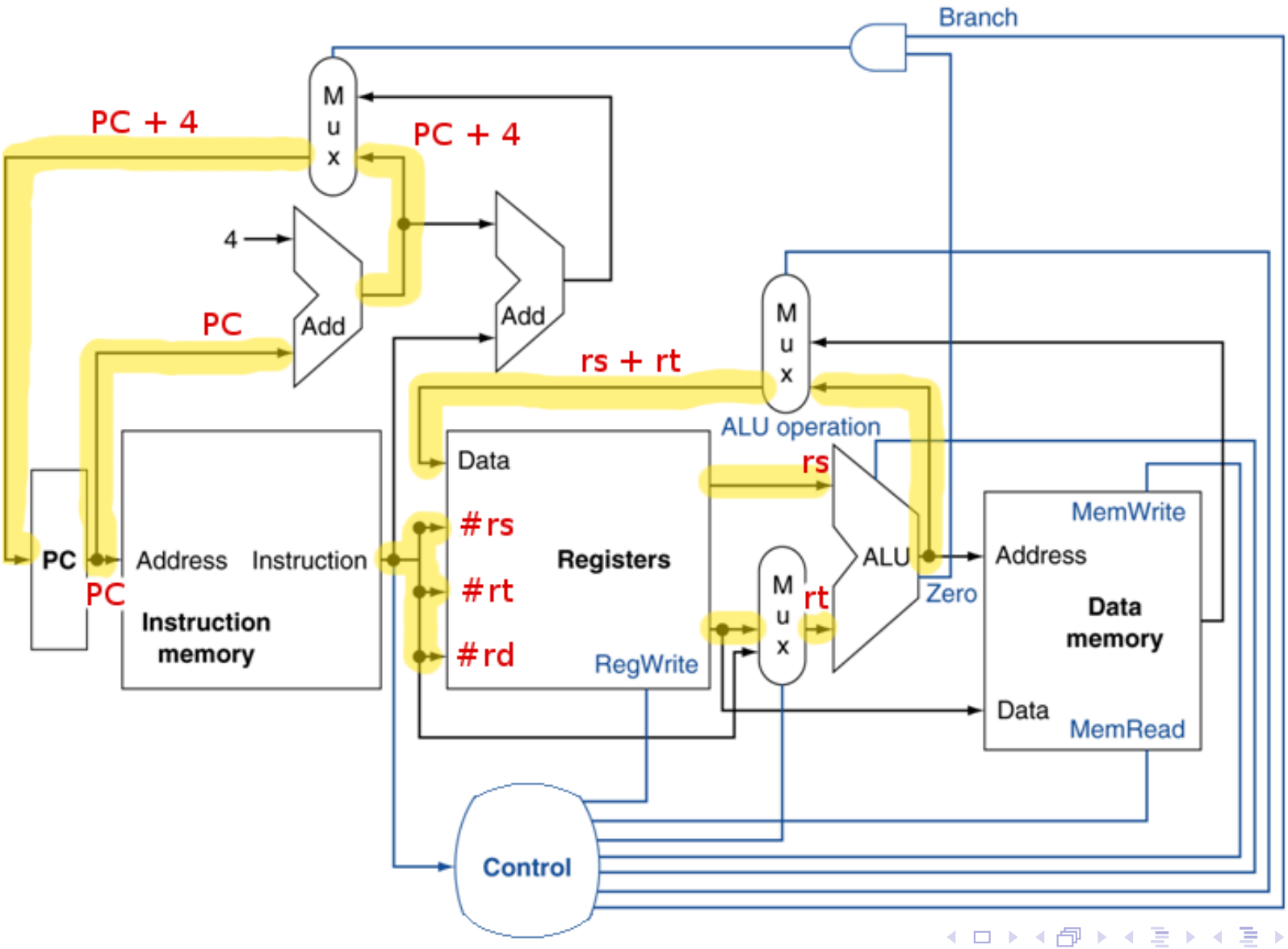


Can't just join wires together $\rightarrow$ use multiplexers

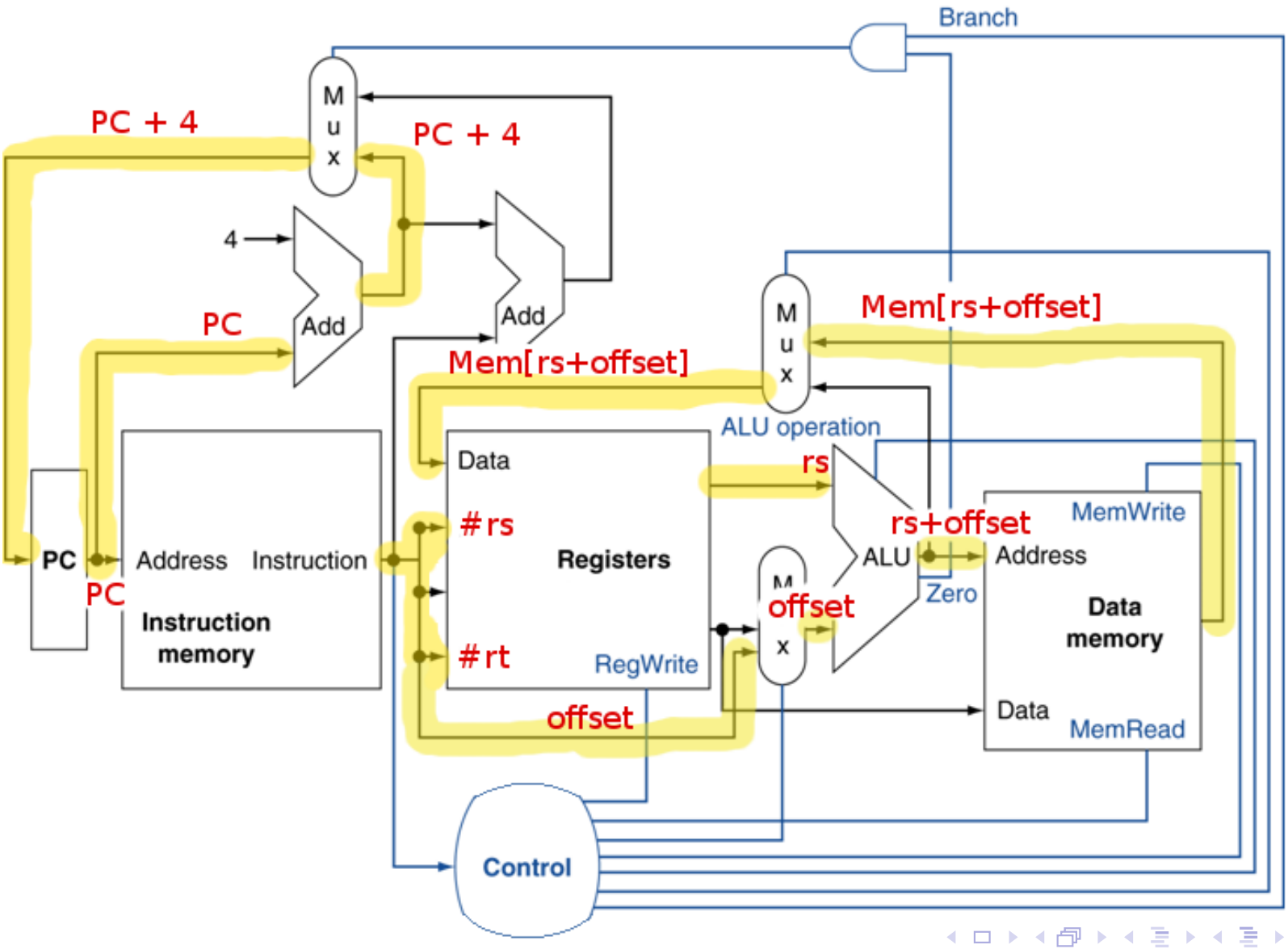
Simplified Single-Cycle CPU



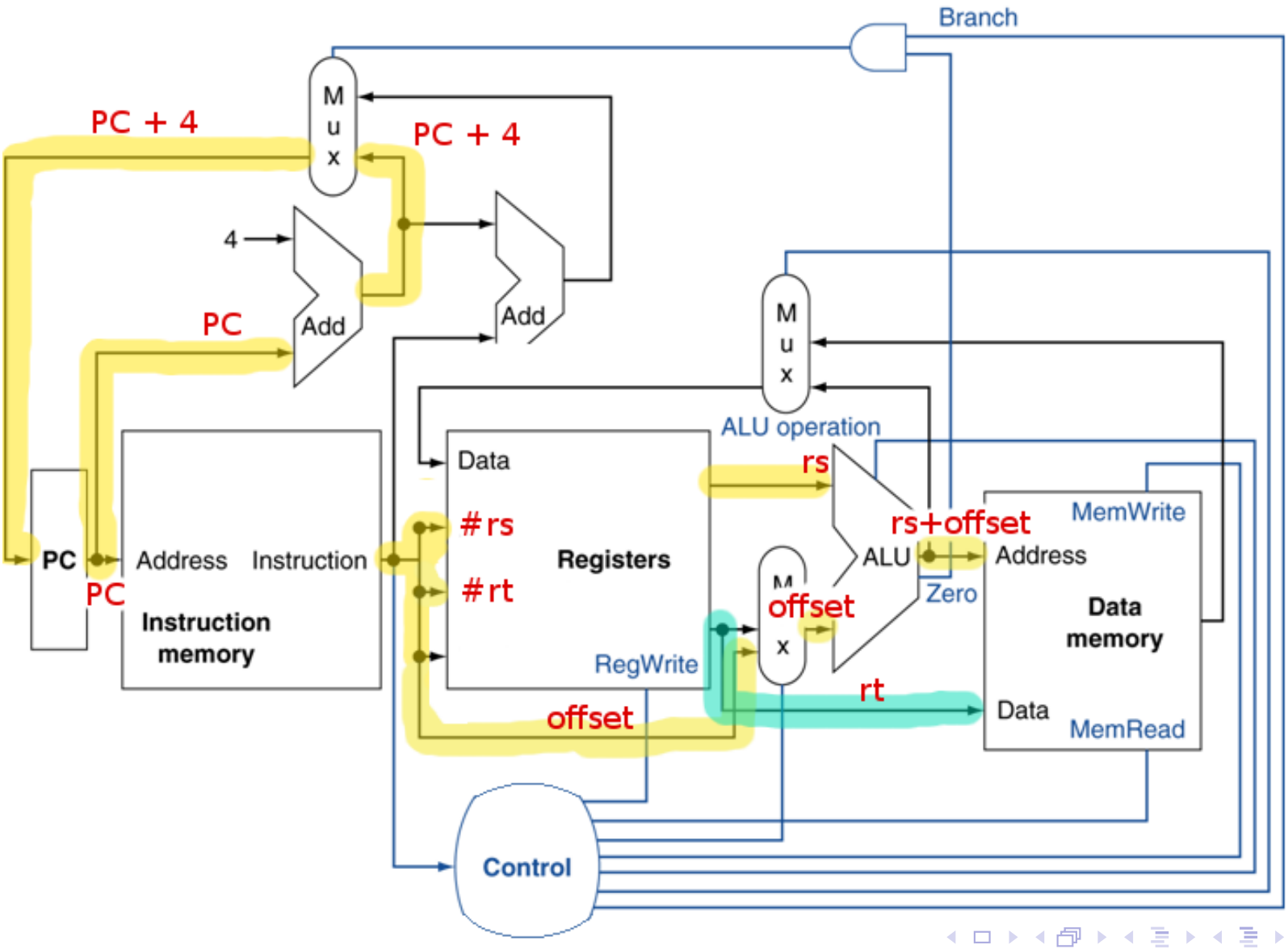
MIPS: add


Branch
M u x
PC + 4
PC + 4
4
Add
Add
PC
rs + rt
M u x
ALU operation
rs
Data
#rs
#rt
#rd
Registers
RegWrite
M u x
rt
ALU
Zero
MemWrite
Address
Data memory
Data
MemRead
PC
Address
Instruction
PC
Instruction memory
Control

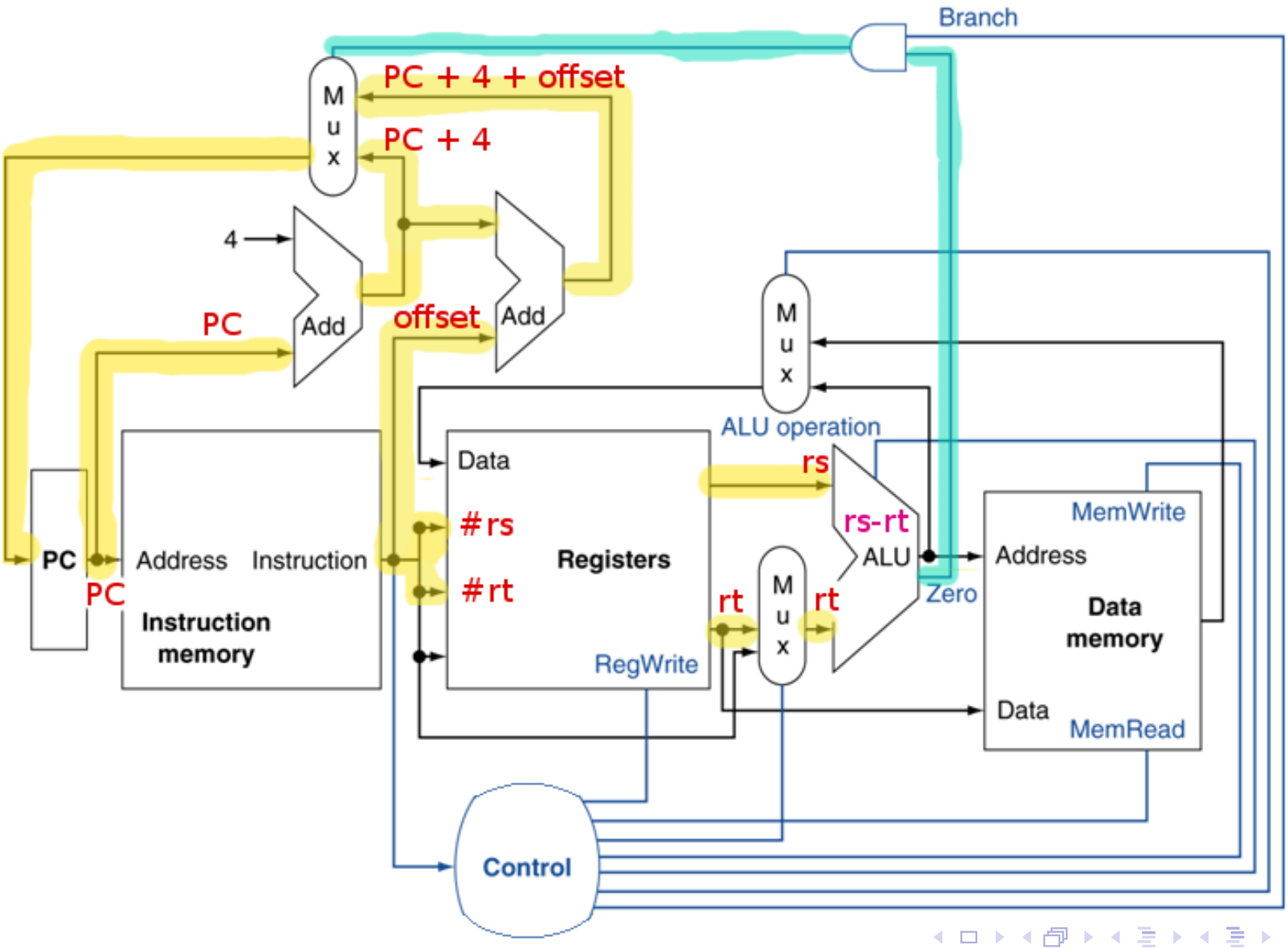
MIPS: lw


Branch
M
u
x
PC + 4
PC + 4
4
Add
Add
PC
Mem[rs+offset]
M
u
x
Mem[rs+offset]
ALU operation
rs
Data
#rs
Registers
rs+offset
MemWrite
ALU
Address
PC
Address
Instruction
PC
Instruction
memory
offset
Zero
Data
memory
#rt
RegWrite
Data
MemRead
offset
Control

MIPS: sw


Branch
M
u
x
PC + 4
PC + 4
4
Add
Add
PC
M
u
x
ALU operation
Data
#rs
#rt
Registers
RegWrite
rs
rs+offset
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
offset
rt
PC
Address
Instruction
PC
Instruction
memory
Control

MIPS: beq

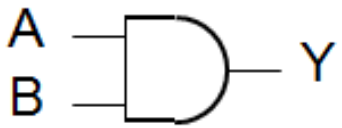

Branch
M
u
x
PC + 4 + offset
PC + 4
4
Add
PC
Add
offset
M
u
x
ALU operation
Data
#rs
#rt
Registers
RegWrite
rs
rs-rt
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
rt
M
u
x
rt
PC
Address
Instruction
PC
Instruction
memory
Control

Logic Design Basics

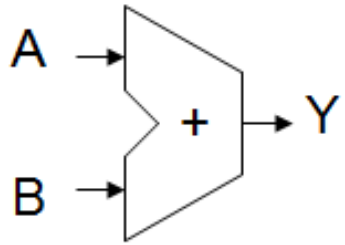
- Information encoded in binary: voltage level $\rightarrow \{0, 1\}$
  - One wire per bit
  - Multi-bit data encoded on multi-wire buses
- Combinational element
  - Operate on data
  - Output is a function of input
- State (sequential) elements
  - Store information

Combinational Elements

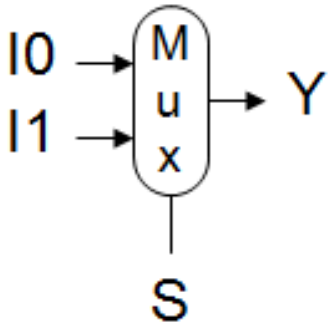
AND-gate: Y = A & B



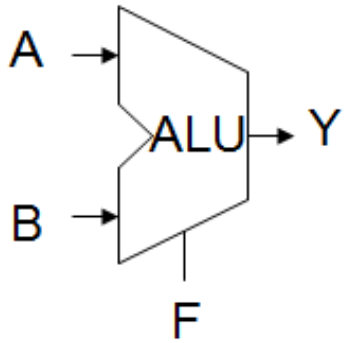
Adder: Y = A + B



Multiplexer: Y = S ? I1 : I0



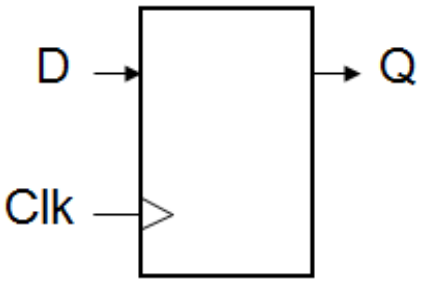
ALU: Y = F(A,B)

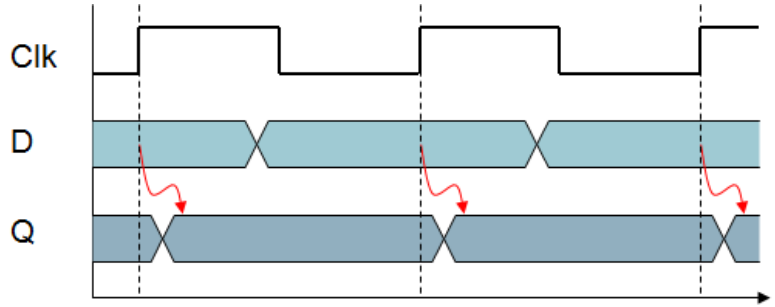


Sequential Elements

- Register: stores data in a circuit

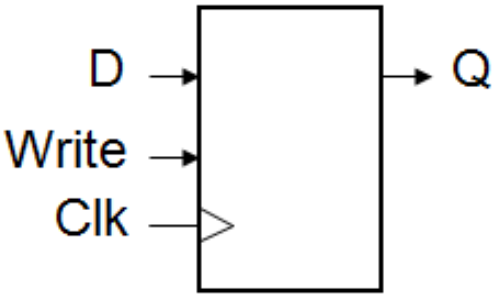
  Uses a clock signal to determine when to update the stored value
- Edge-triggered: update when Clk changes from 0 to 1

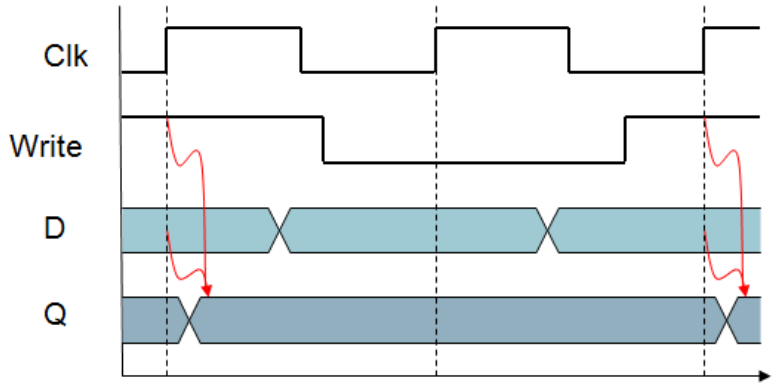




Sequential Elements

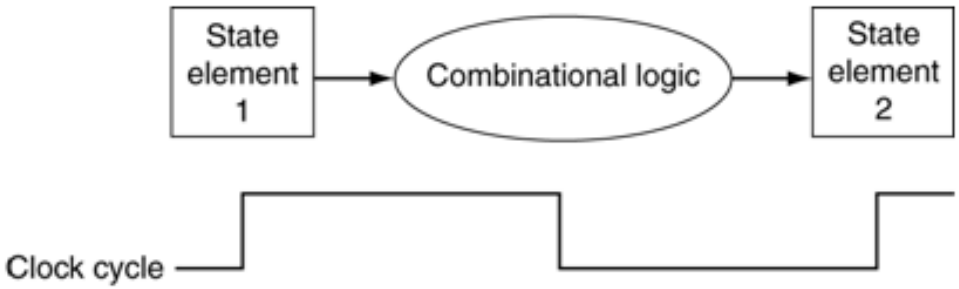
- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later





Clocking Methodology

- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - Longest delay determines clock period





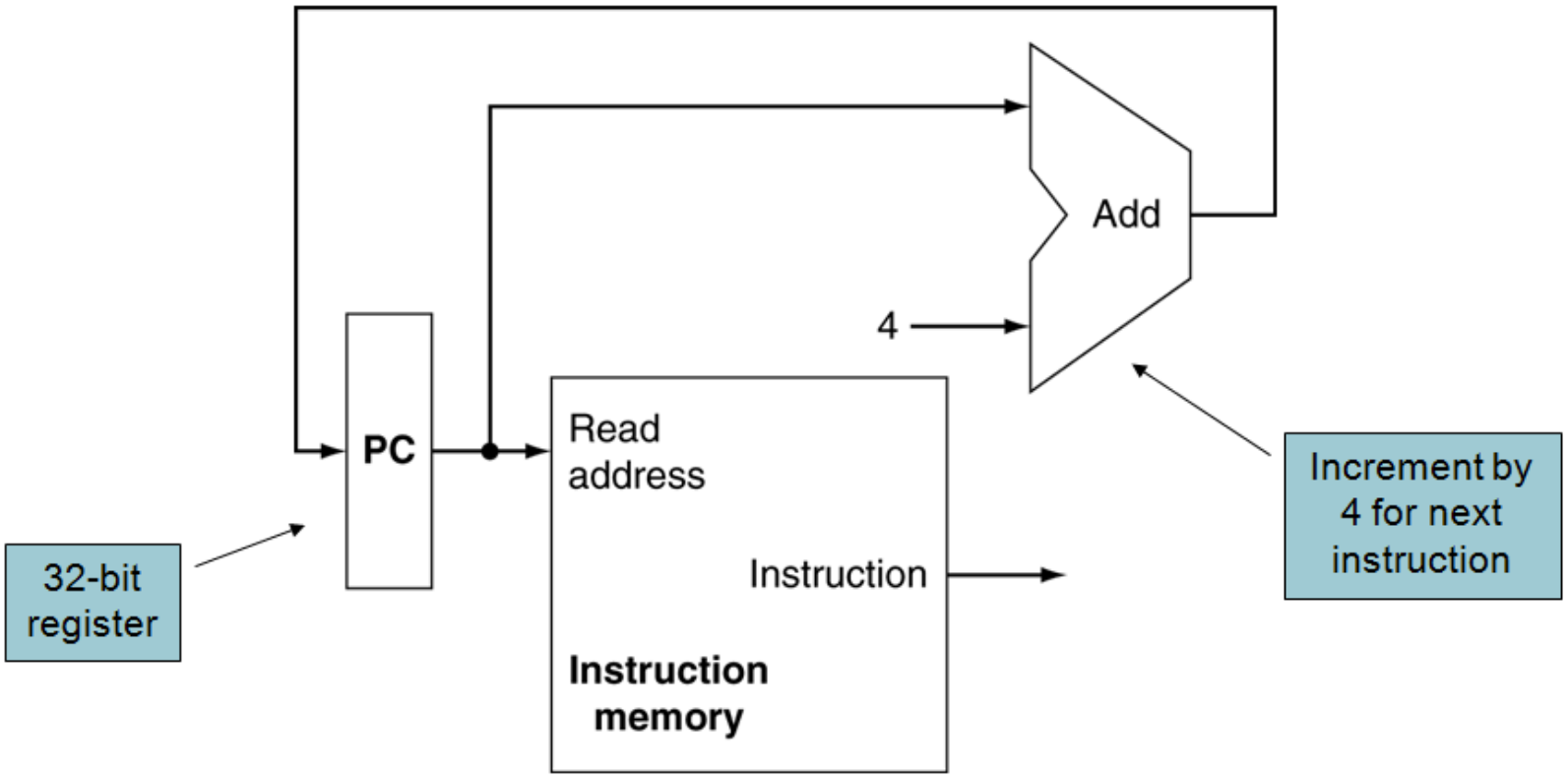
Building a Datapath

- Datapath
  - Elements that process data and addresses in the CPU

    Registers, ALUs, mux's, memories, ...
- Simplified MIPS datapath:

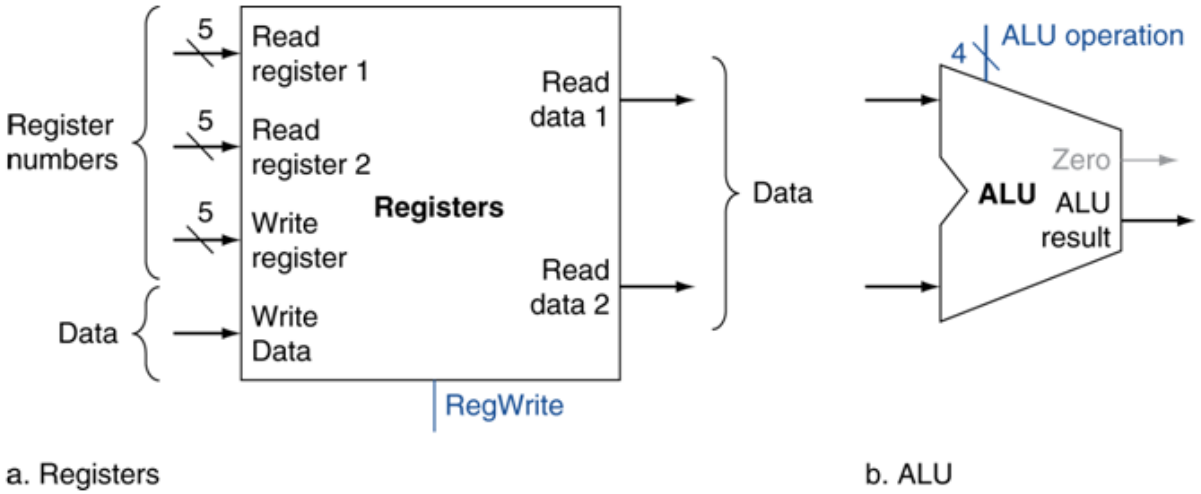
  Refine the overview

Instruction Fetch



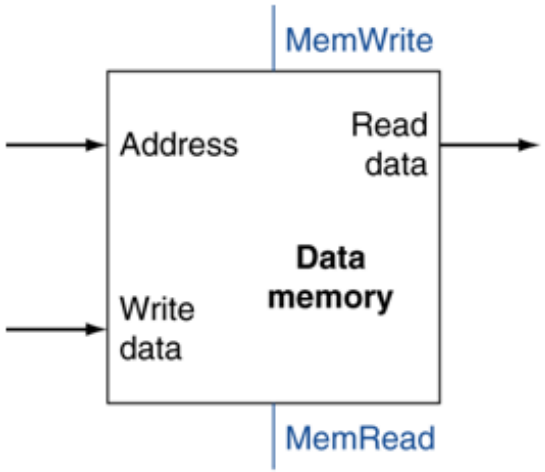
R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

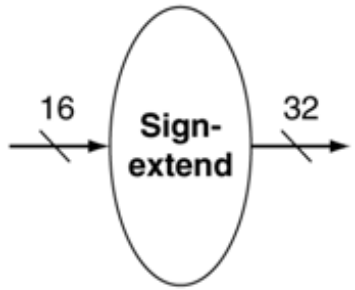


Load/Store Instructions

- Read register operands
- Calculate address: rs + offset
  - Do sign-extension 16-bit offset
  - ALU
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit

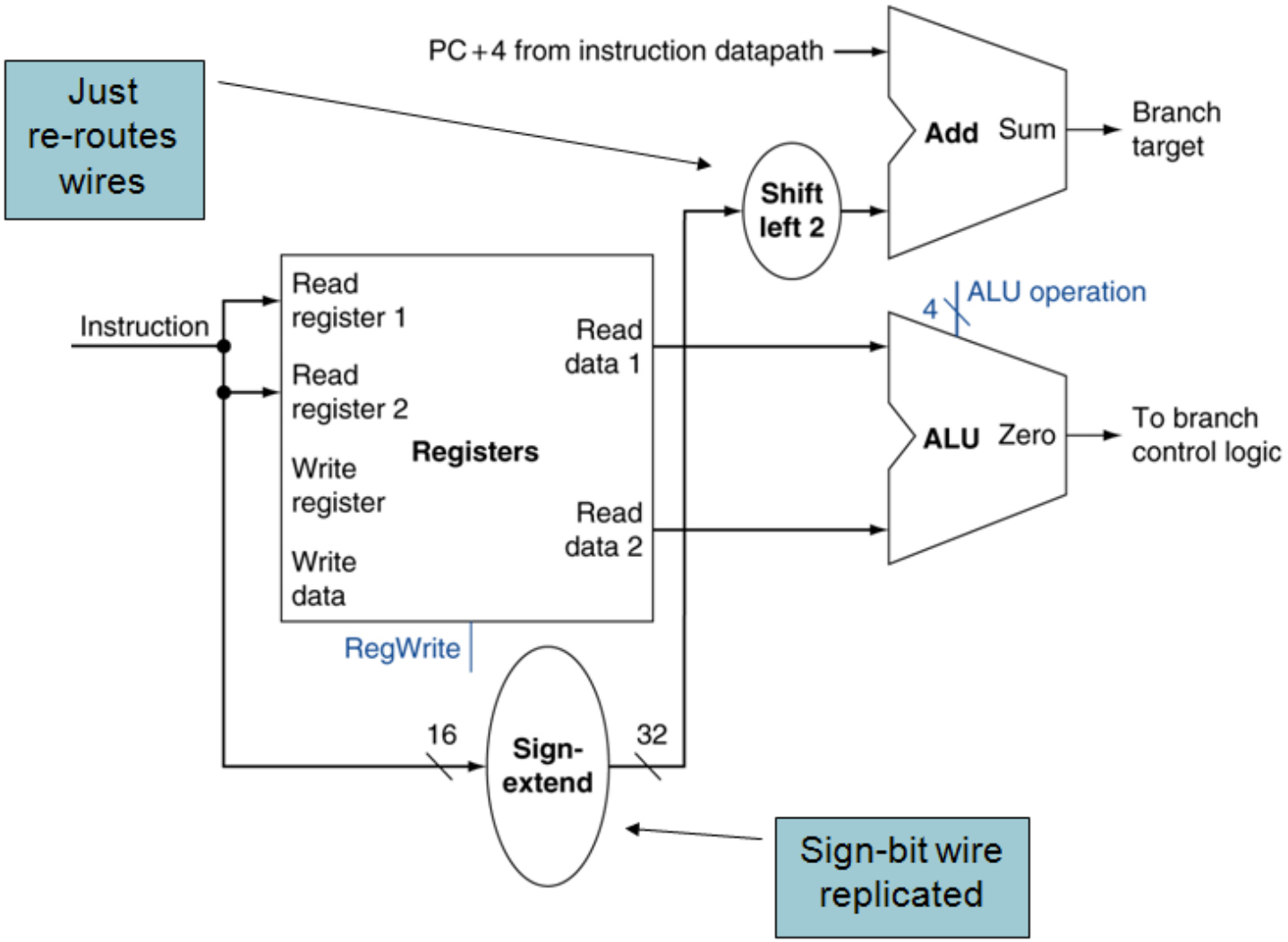


b. Sign extension unit

Branch Instructions

- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
- Sign-extend displacement
- Shift left 2 places (word displacement)
- Add to PC + 4
  - Already calculated by instruction fetch

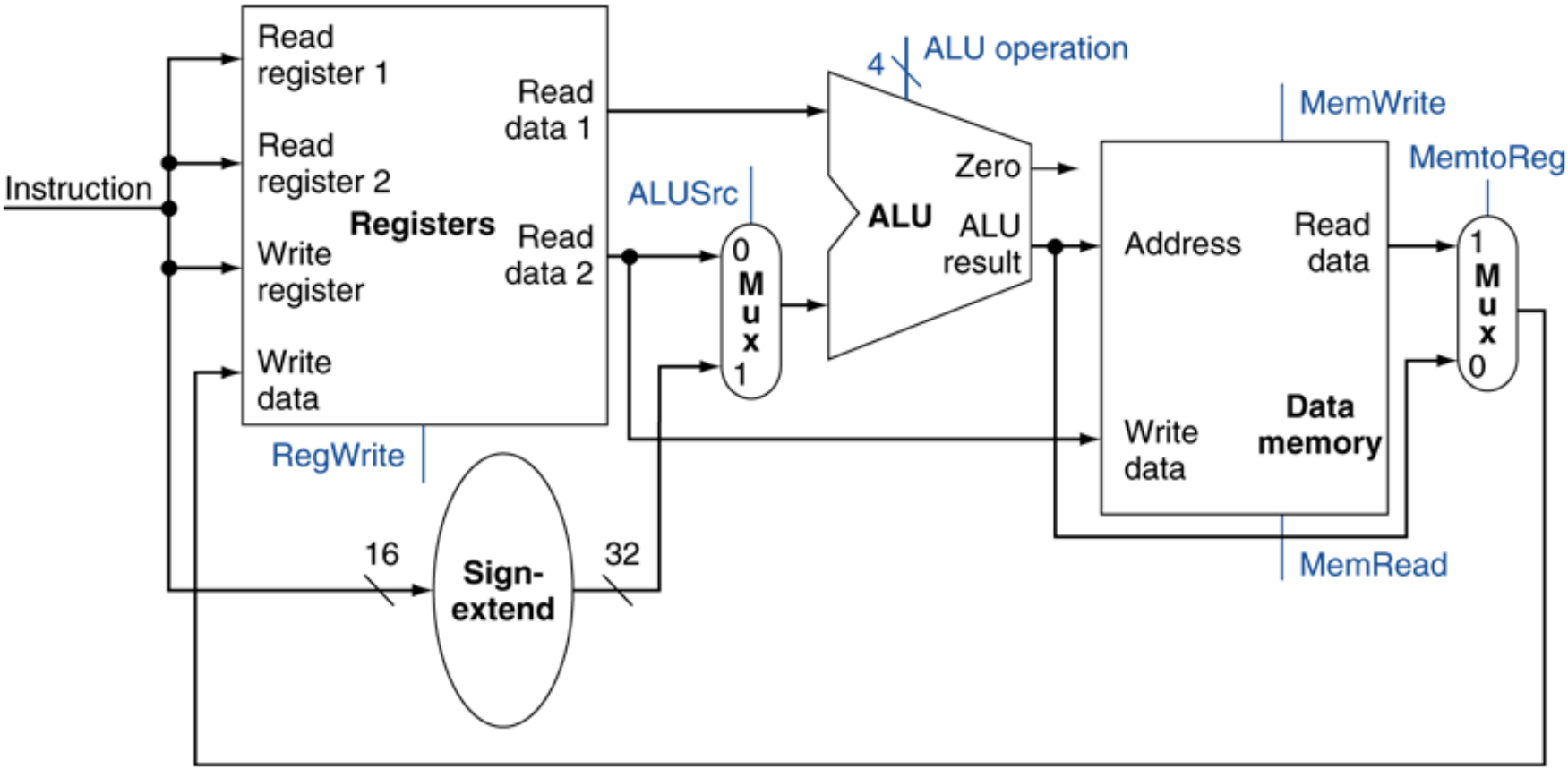
Branch Instructions



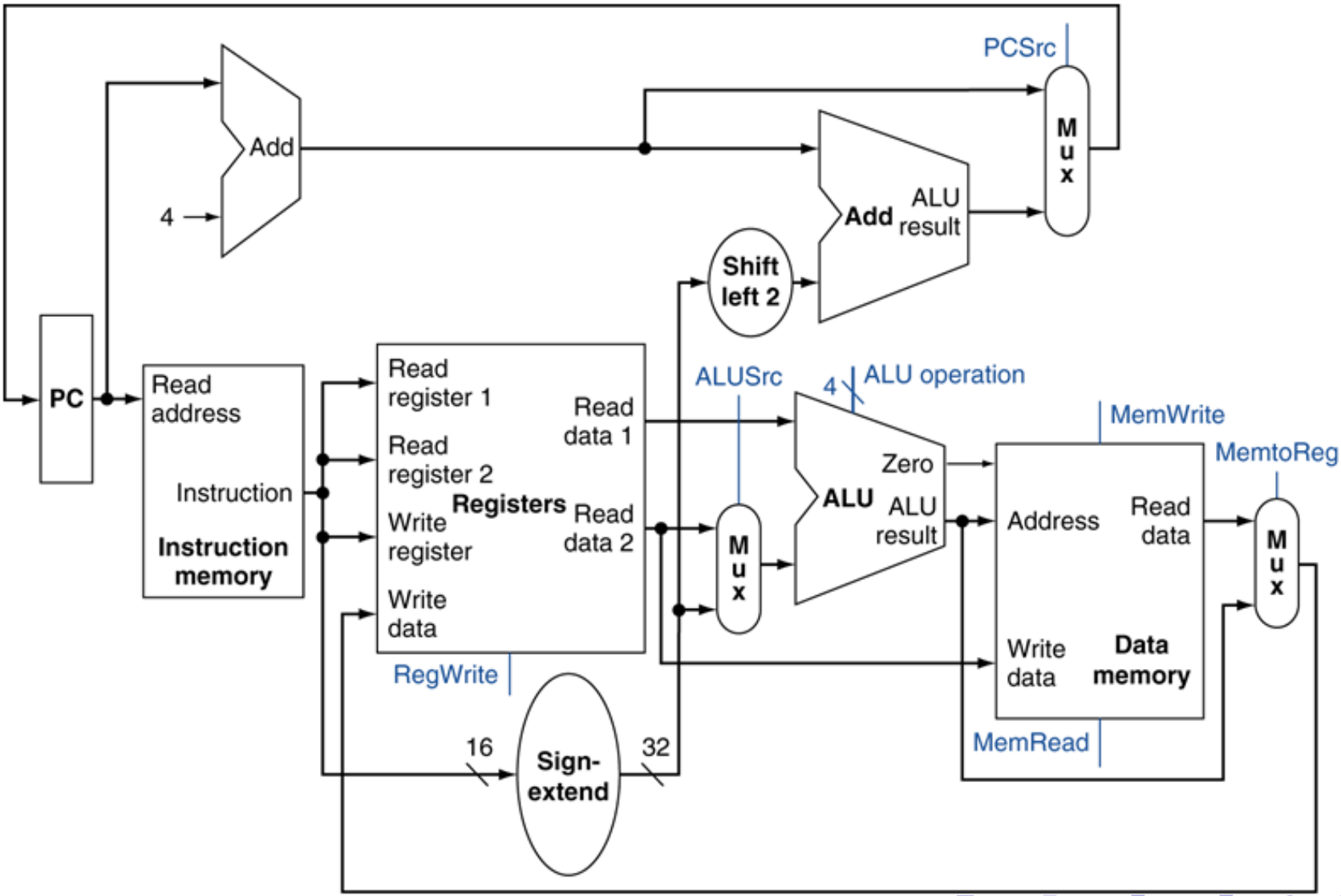
Composing the Elements

- First-cut data path does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, we need separate instruction and data memories (Memory hierarchy: instruction cache and data cache)
- Use multiplexers where alternate data sources are used for different instructions

R-Type/Load/Store Datapath



Full Datapath



ALU Control: Mapping Instructions to What we want

ALU used for

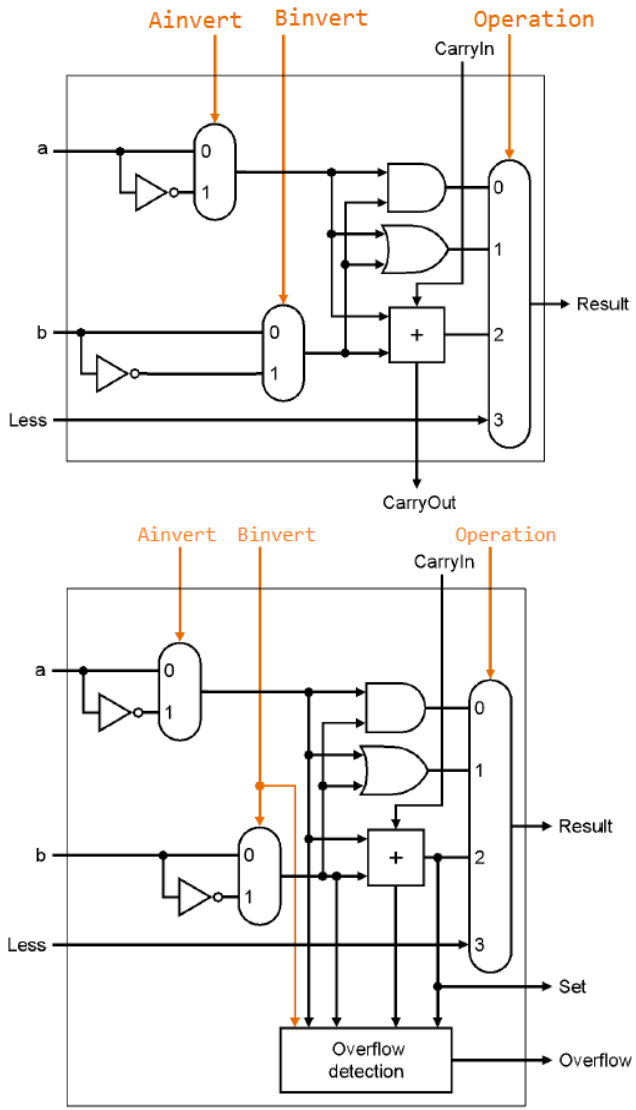
- Load/Store: F = add
- Branch: F = subtract
- R-type: F depends on funct field

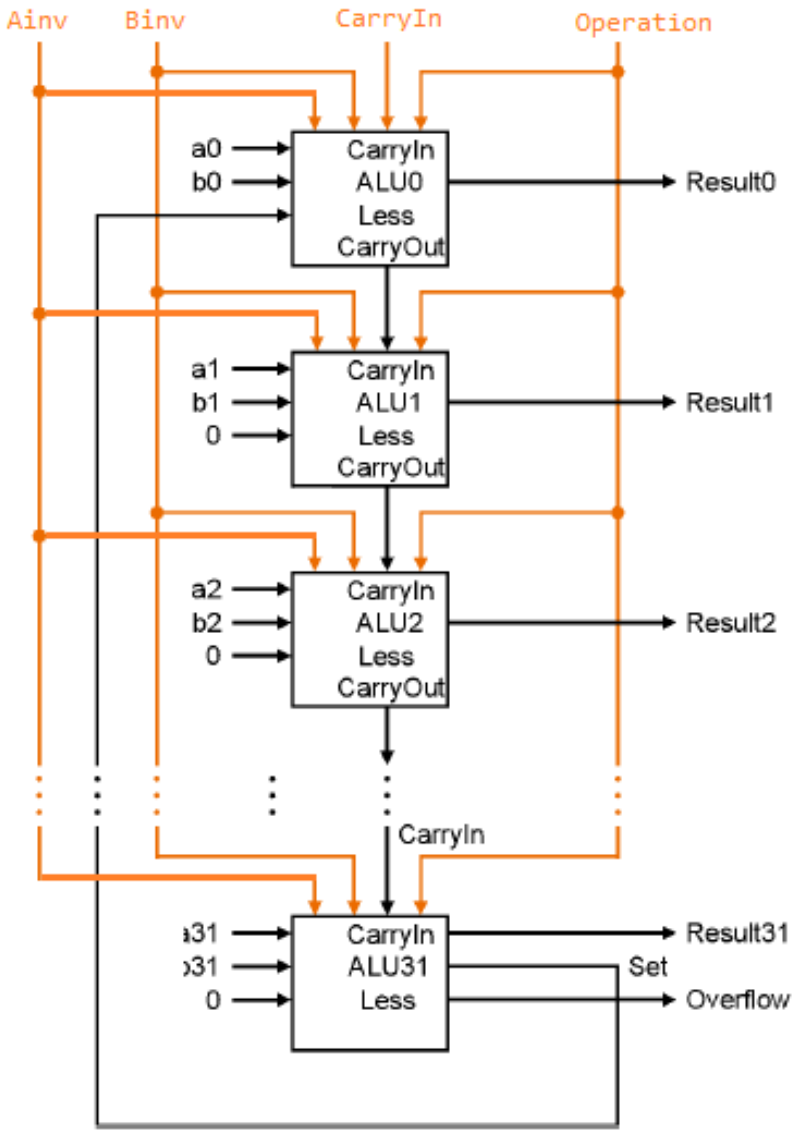
funct	funct in binary	ALU Function
0x20	[100000]	add
0x22	[100010]	sub
0x24	[100100]	and
0x25	[100101]	or
0x2A	[101010]	slt

ALU Control: ALU Behavior



Control	Function
[0000]	and
[0001]	or
[0010]	add
[0110]	subtract
[0111]	slt
[1100]	nor





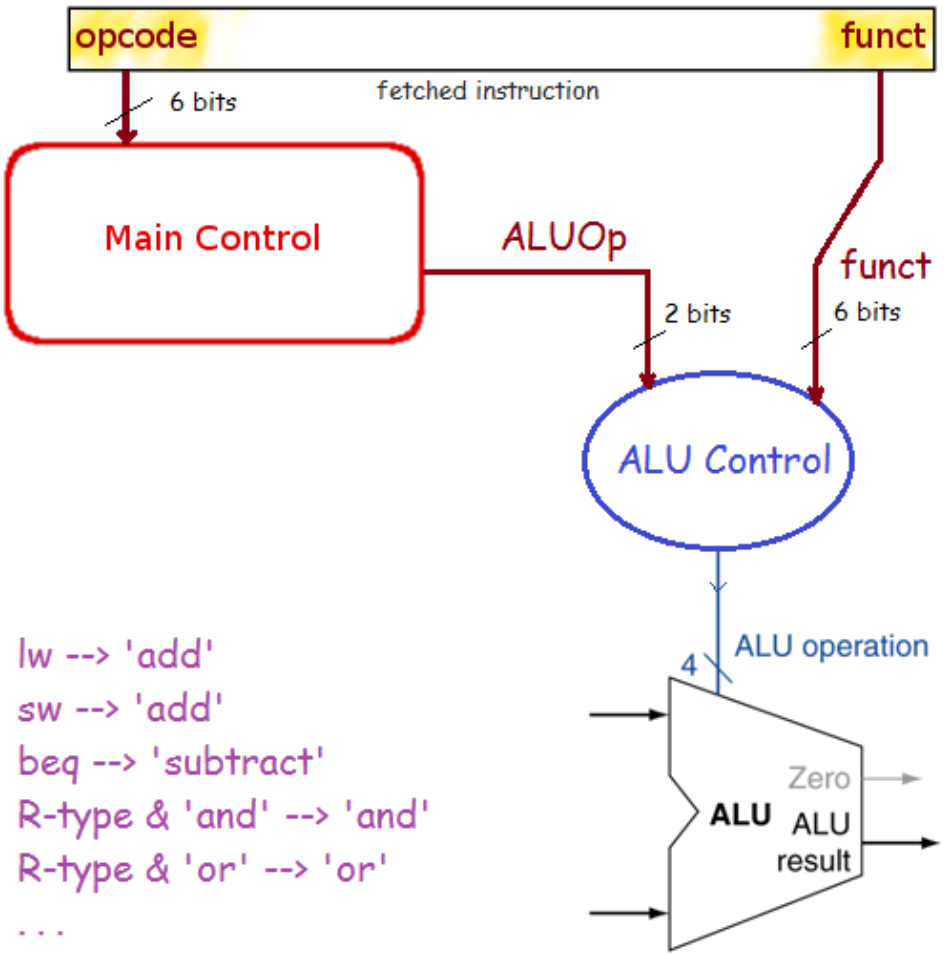
Mapping Machine Code to ALU Control

opcode	mode	operation	funct	ALU function	ALU control
lw	[00]	load word	[xxxxxx]	add	[0010]
sw	[00]	store word	[xxxxxx]	add	[0010]
beq	[01]	branch equal	[xxxxxx]	subtract	[0110]
R-type	[10]	add	[100000]	add	[0010]
		subtract	[100010]	subtract	[0110]
		AND	[100100]	AND	[0000]
		OR	[100101]	OR	[0001]
		set-on-less-than	[101010]	slt	[0111]

- Assume 2-bit mode derived from opcode

  We will call it “ALUOp”

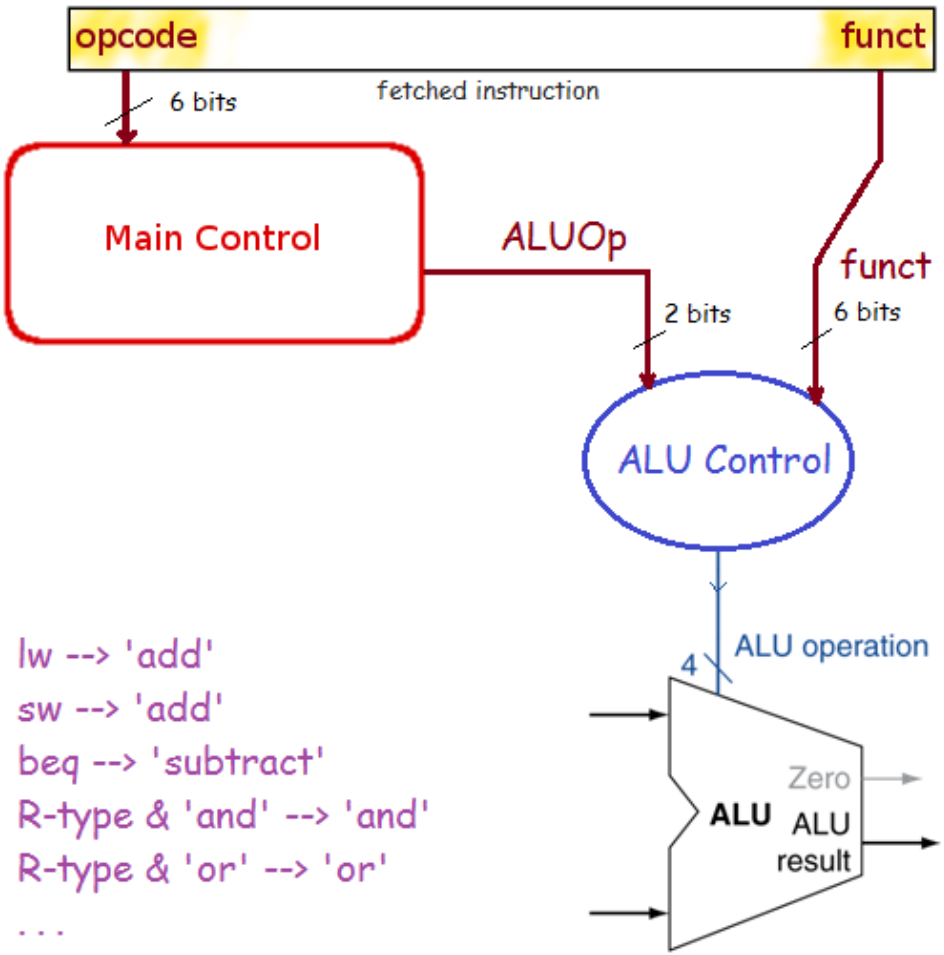
ALU Control



lw --> 'add'
sw --> 'add'
beq --> 'subtract'
R-type & 'and' --> 'and'
R-type & 'or' --> 'or'
...

ALU Control Truth Table

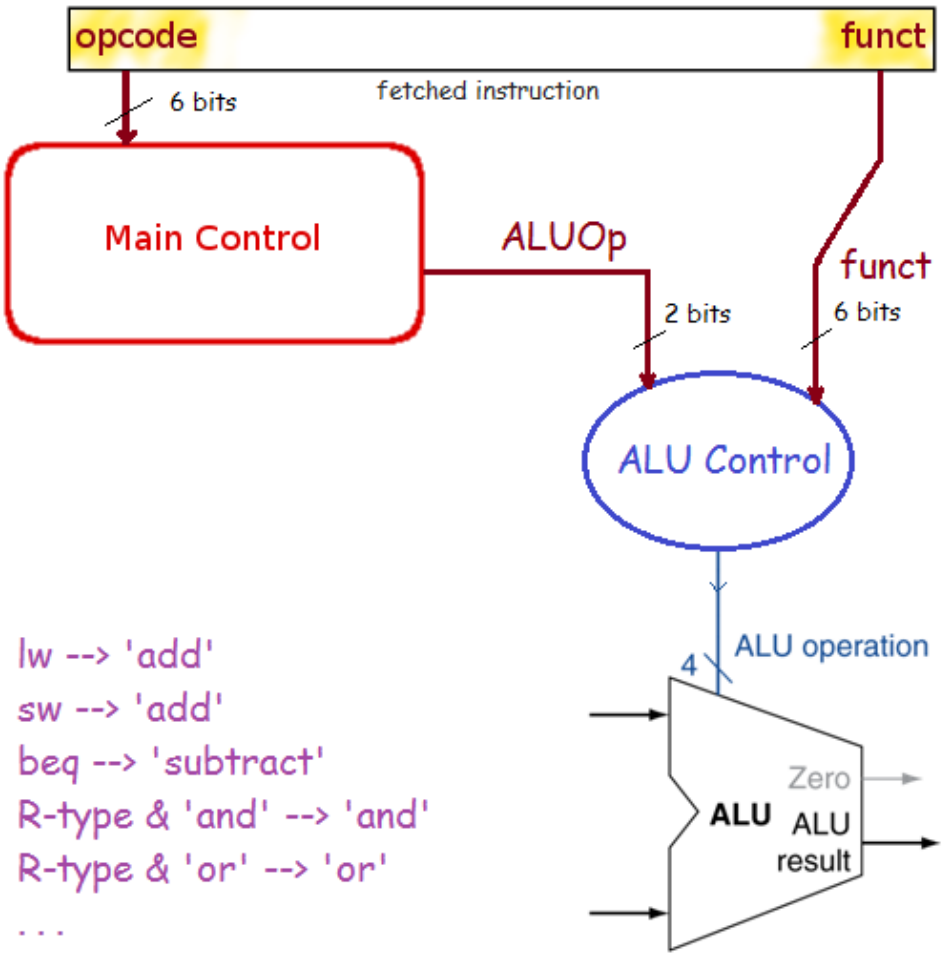
ALU Control



ALU Control Truth Table

	Input	Output
	[ALUop,funct]	ALU Operation
lw	[00,xxxxxx]	[0010]

ALU Control

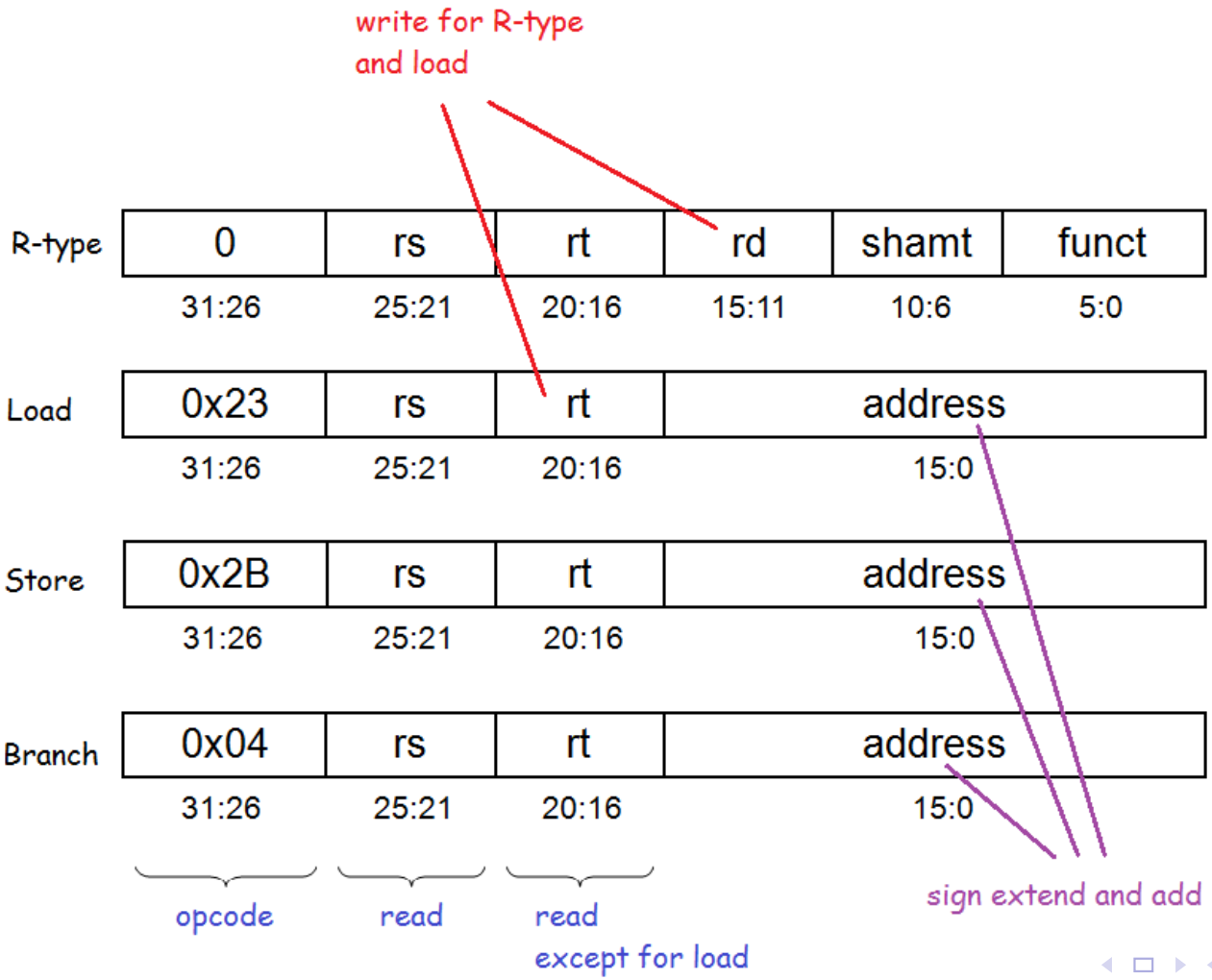


ALU Control Truth Table

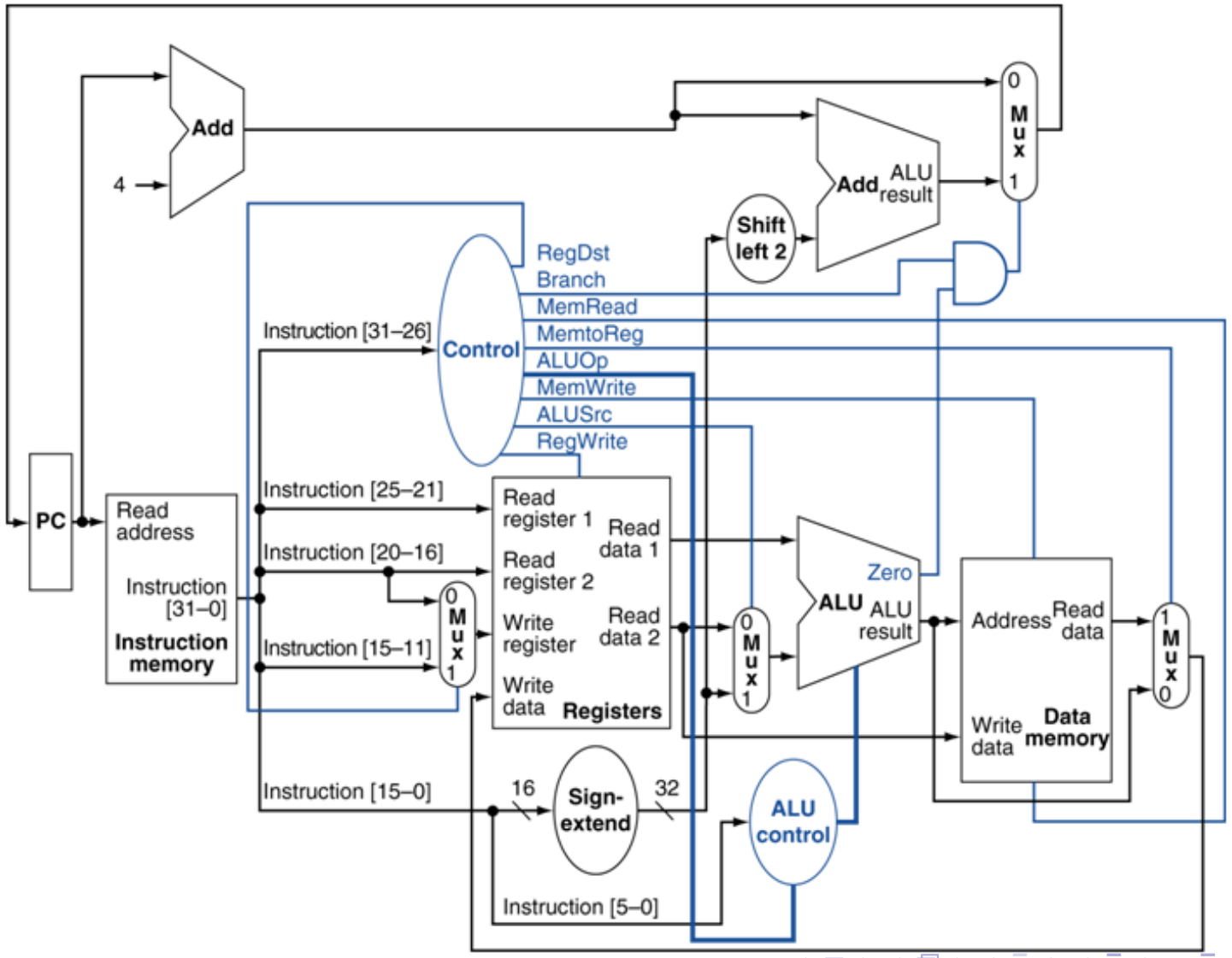
	Input	Output
	[ALUop,funct]	ALU Operation
lw	[00,xxxxxx]	[0010]
sw	[00,xxxxxx]	[0010]
beq	[01,xxxxxx]	[0110]
add	[10,100000]	[0010]
sub	[10,100010]	[0110]
and	[10,100100]	[0000]
or	[10,100101]	[0001]
slt	[10,101010]	[0111]

Main Control Unit

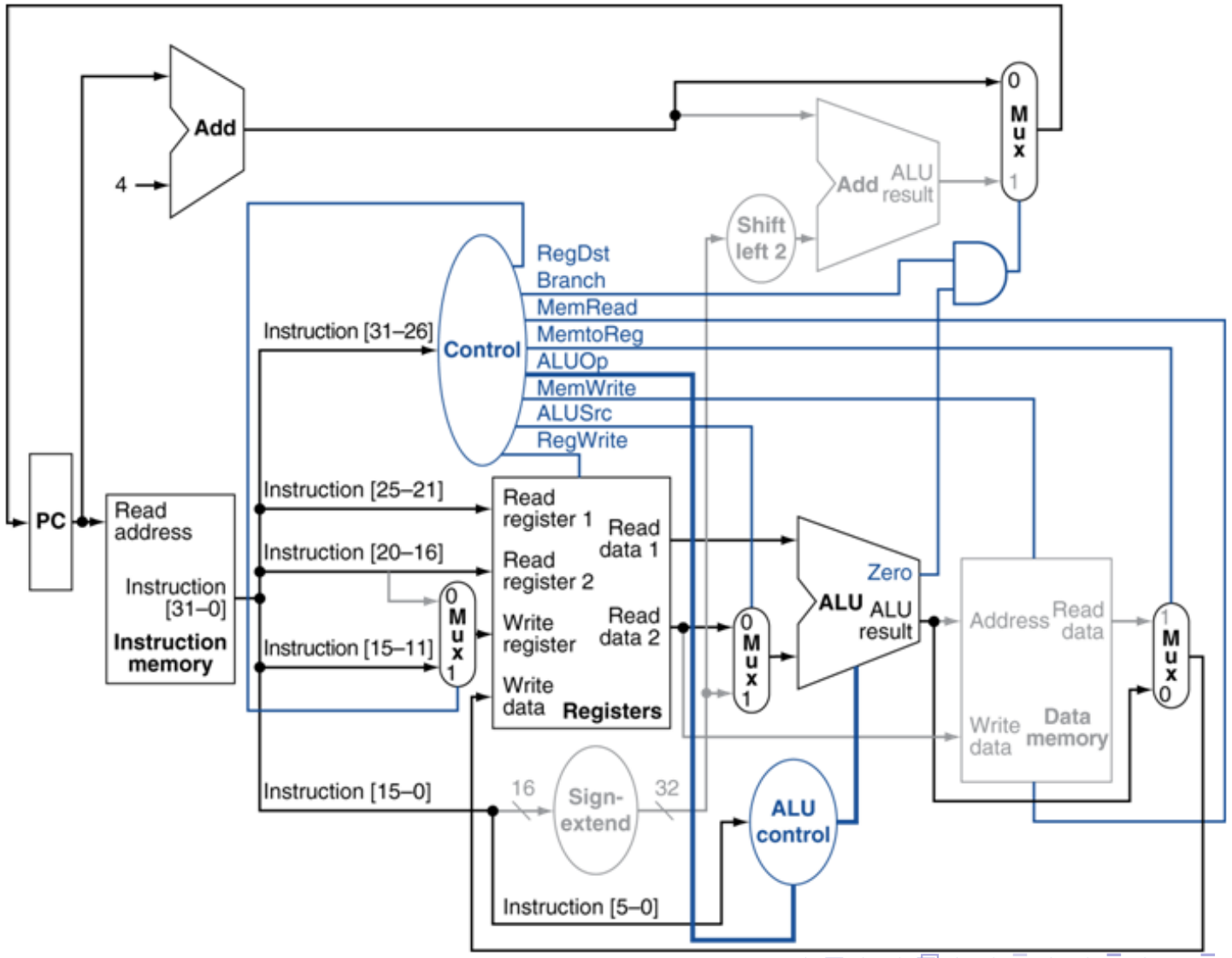
- Map from instruction to control signals



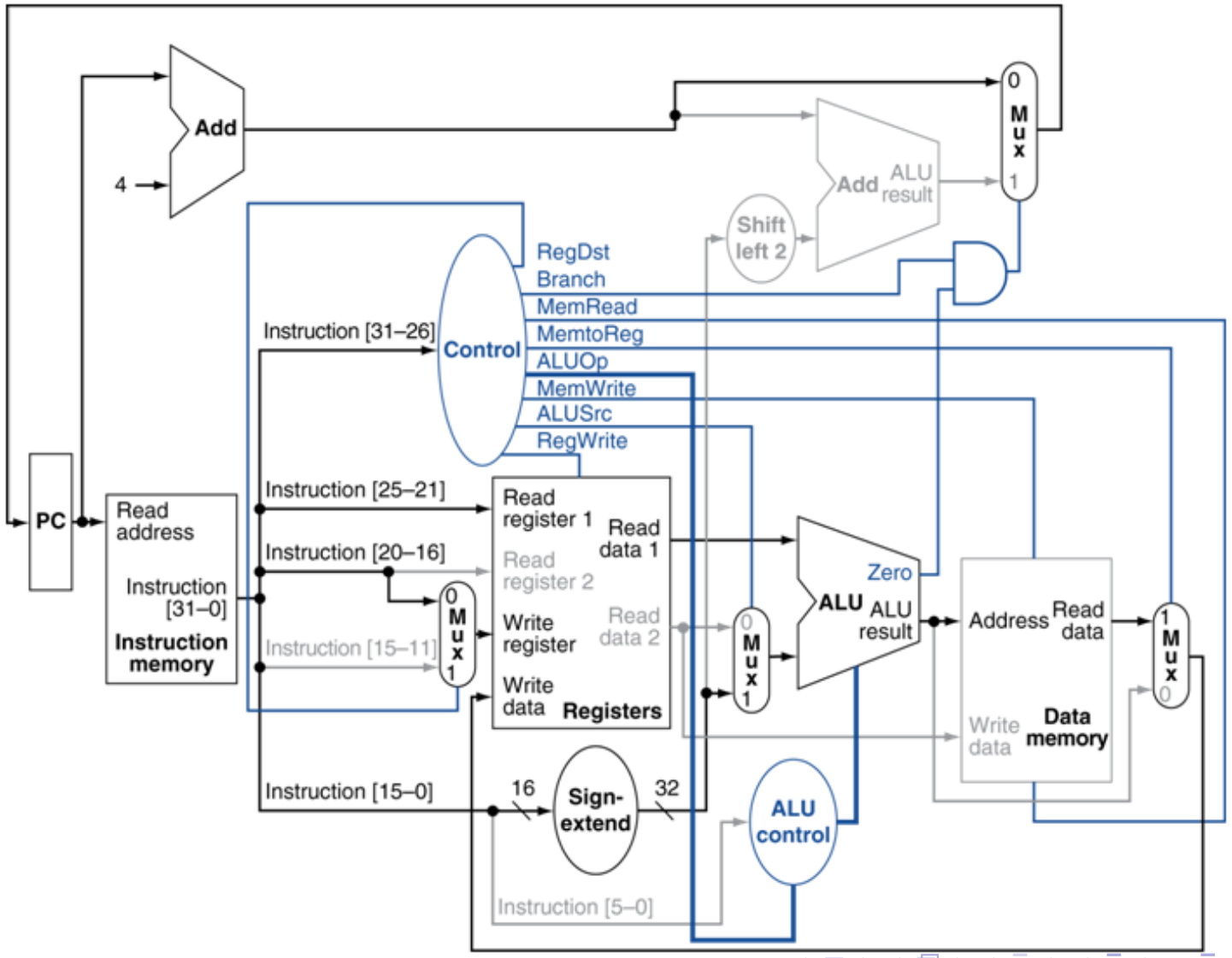
Datapath with Control



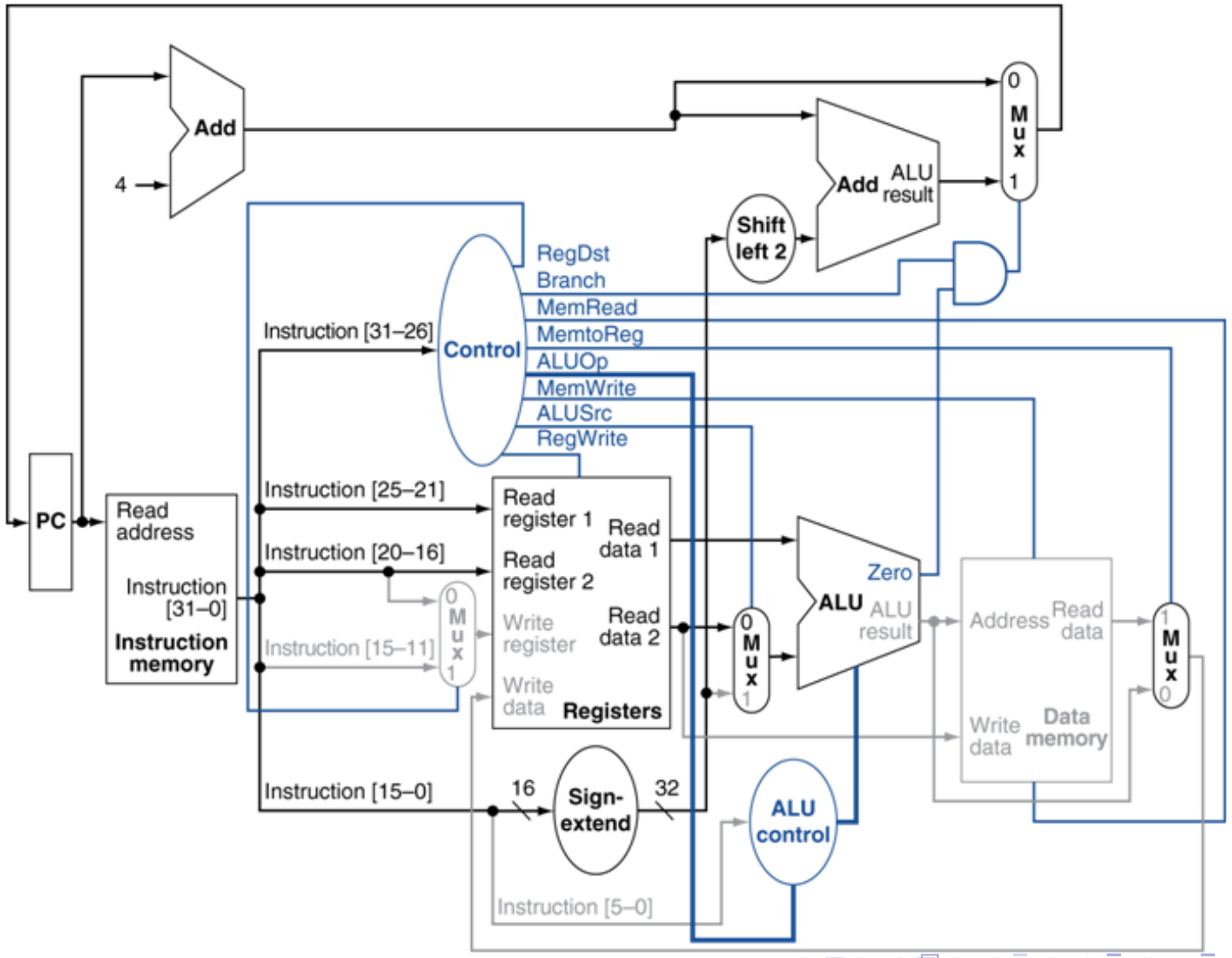
R-Type Datapath



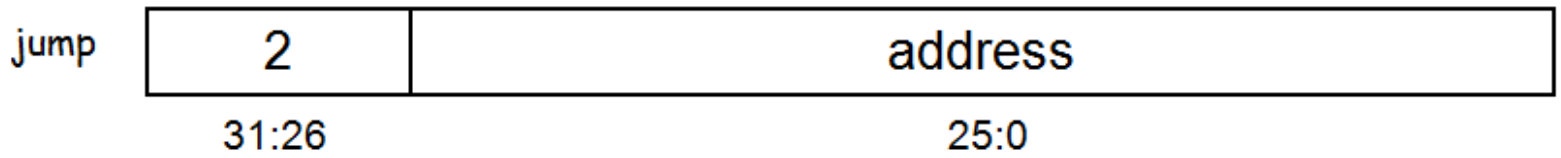
Load Datapath



Branch-on-Equal Datapath

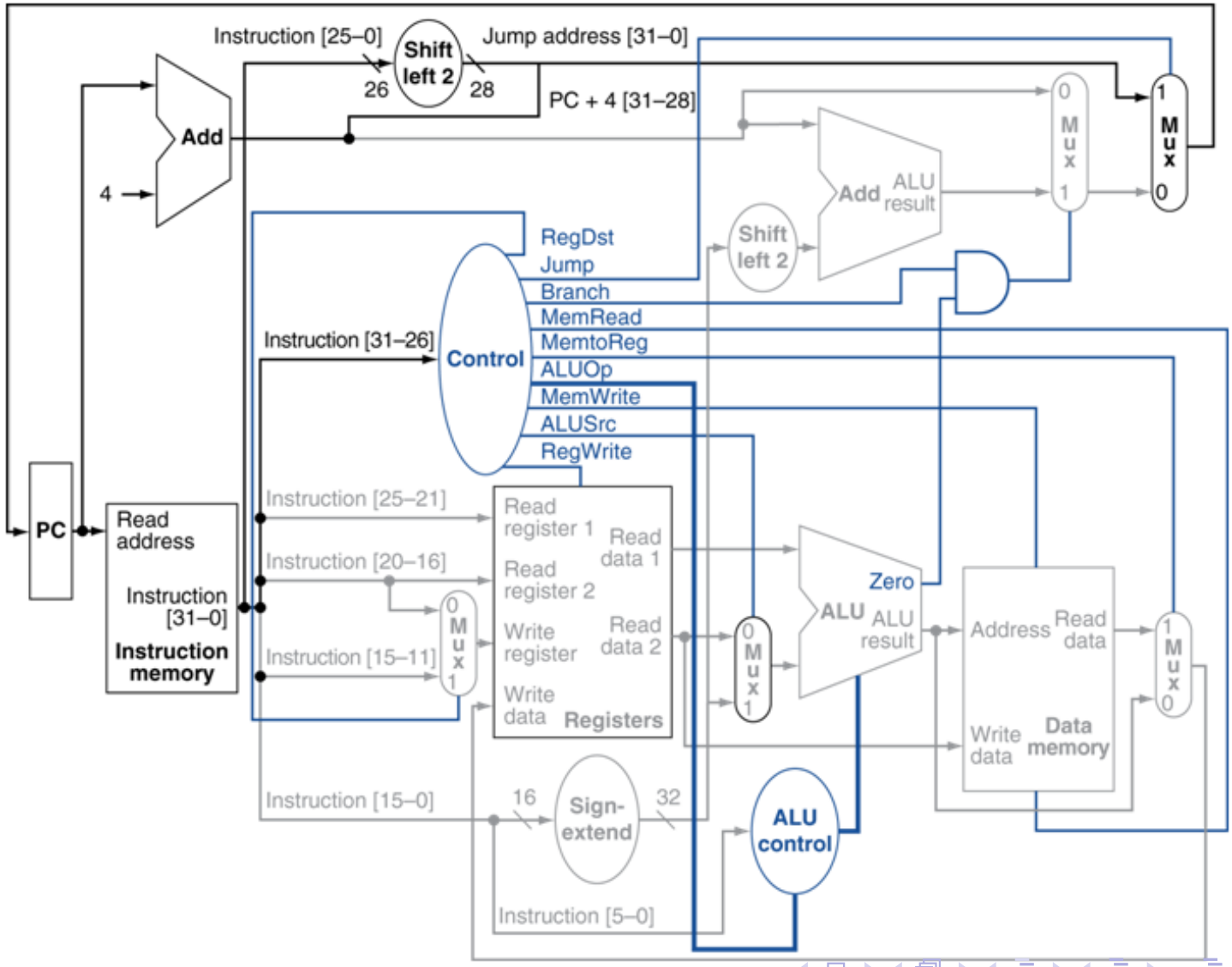


Implementing Jumps



- Jump uses word address
- Update PC with concatenation of
  - Top 4 bits of old PC
  - 26-bit jump address
  - 00
- Need an extra control signal decoded from opcode

Datapath with Jump Added



Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction

  Instruction memory $\rightarrow$ register file $\rightarrow$ ALU $\rightarrow$ data memory $\rightarrow$ register file
- Not feasible to vary period for different instructions
- Violates design principle

  Making the common case fast
- Improve upon single-cycle machine
  - multi-cycle machine
  - pipelining

Closing note

- อ่าน P&H5e, บทที่ 4.
- อ่าน บทที่ 5 เอกสารประกอบรายวิชา 188 322 สถาปัตยกรรมคอมพิวเตอร์, COA notes (April 25, 2014).

